



Rapport de stage

Numérisation de marégrammes

GEGADEN Valentin

Encadré par

BOUWMANS Thierry (MIA), DJEUMENI NOUBISSIE Lucrèce (LIENSs) et PÉLISSIÉ De MONTEMONT Thomas (MIA)

Table des matières

1.	Introduction.....	3
1.1.	Présentation de l'entreprise	3
1.2.	Contexte du projet.....	4
1.3.	Objectif du stage	5
2.	État et analyse du problème	5
2.1.	Présentation des données.....	5
2.2.	Défis techniques	7
2.3.	Énumération des étapes pour mener à bien ce projet	8
3.	Mise en oeuvre.....	8
3.1.	Détails de la mise en oeuvre	8
3.1.1.	Identifier la date	8
3.1.2.	Extraire la grille.....	9
3.1.3.	Extraire les courbes	10
3.1.4.	Trouver le point de départ.....	12
3.1.5.	Gérer les croisements.....	14
3.1.6.	Gérer le suivi de la courbe du jour suivant.....	16
3.1.7.	Convertir chaque valeur en un temps et une hauteur d'eau	17
3.2.	Cas limites.....	18
4.	Conclusion	20
4.1.	Bilan des apports du projet	20
4.2.	Évolution du projet	21
4.3.	Bilan personnel.....	23
4.4.	Ouverture	23

1. Introduction

1.1. Présentation de l'entreprise

Ce stage a été réalisé au sein de La Rochelle Université, plus précisément sous l'égide du laboratoire MIA (Mathématiques, Image et Applications) et du laboratoire LIENSs (Littoral, ENvironnement et Sociétés) et a été mené grâce à des données du Shom (Service hydrographique et océanographique de la Marine).

Le laboratoire MIA est un laboratoire de La Rochelle Université qui allie expertise des mathématiques et de l'informatique. Il est également membre de la fédération de recherche CNRS MIREs (Centre National de la Recherche Scientifique Mathématiques et interactions, Images et information numérique, REseaux et Sécurité). Il regroupe une vingtaine de membres dont des enseignants-chercheurs et des doctorants. Les travaux effectués par le laboratoire sont majoritairement liés à l'analyse et le traitement de données visuelles ainsi qu'à la modélisation mathématique pour répondre à des problèmes concrets. Le laboratoire a pu collaborer avec de nombreuses entreprises et institutions renommées telles que Dassault (aviation), Bull (systèmes et logiciels informatiques) ou encore l'INA (Institut National de l'Audiovisuel).

La laboratoire LIENSs est un autre laboratoire rattaché à La Rochelle Université. Il est lui aussi sous la tutelle du CNRS et ses recherches sont plus orientées sur le fonctionnement de la zone littorale, son évolution au fil du temps et en particulier dans le contexte d'aujourd'hui avec le changement global et d'urbanisation croissante des côtes, son usage et exploitation durable. Le laboratoire compte bon nombre de chercheurs, doctorants et collaborateurs répartis entre plusieurs équipes dont chacune à sa spécialité. En effet, l'approche du laboratoire est multidisciplinaire et couvre un large spectre de domaines allant des sciences de l'environnement aux sciences humaines en passant par la chimie et les biotechnologies. Le laboratoire a notamment pu collaborer avec Total (énergie), Léa Nature (cosmétiques et produits bio) ou encore avec les sites du Muséum Nationale d'Histoire Naturelle de Paris et Dinard.

Le Shom est un opérateur public placé sous la tutelle du ministère des Armées dont la spécialité est l'information géographique maritime et littorale. Son rôle est de connaître, décrire et prévoir l'environnement physique marin. Des centaines de données maritimes et littorales sont disponibles sur leur site internet. Le siège du Shom est à Brest mais il collabore avec une grosse partie des départements de France (métropolitaine et d'outre-mer) et des régions. Il travaille également avec Météo France ou des universités comme celle de La Rochelle. Les données utilisées lors de ce stage sont des données provenant du port de Douala au Cameroun.

1.2. Contexte du projet

Lorsque l'on regarde la frise chronologique des marégrammes numérisées du port de Douala (cf Figure 1), on voit tout de suite qu'il y a beaucoup de marégrammes scannés mais non numérisés.

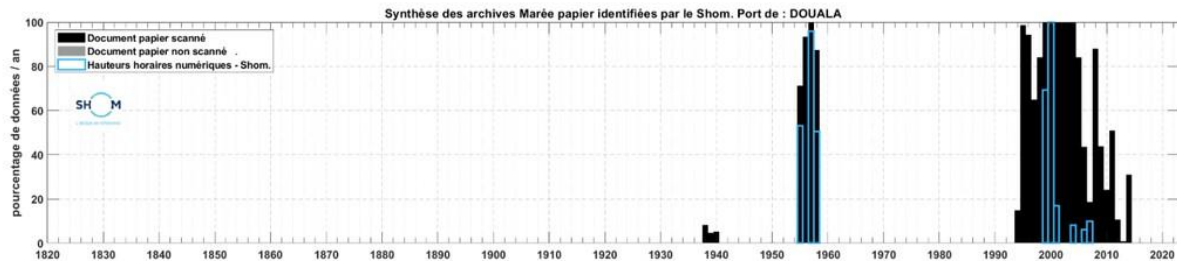


Figure 1 – Graphique montrant la part de marégrammes scannés et numérisés du port de Douala (Source : <https://refmar.shom.fr>)

Mais concrètement, qu'est-ce qu'un marégramme ? Un marégramme est un papier sur lequel une ou plusieurs courbes sont tracées et qui va représenter un niveau d'eau. Il existe des marégrammes dont la longueur est d'une semaine et d'autre d'un jour. D'où le fait que plusieurs courbes peuvent se retrouver sur un marégramme. Par gain de papier, on laisse le stylo tracer plusieurs jours sur un marégramme de seulement 24h de long.

Un marégramme est tracé par un marégraphe dont le fonctionnement est simple. Le papier vierge va être enroulé tout autour d'un tube qui va effectuer un tour complet au bout de 24h (pour les marégrammes représentant 24h). Un stylo va continuellement être en contact avec le papier et ainsi tracer les courbes d'hauteur d'eau en fonction du niveau d'eau détecté.

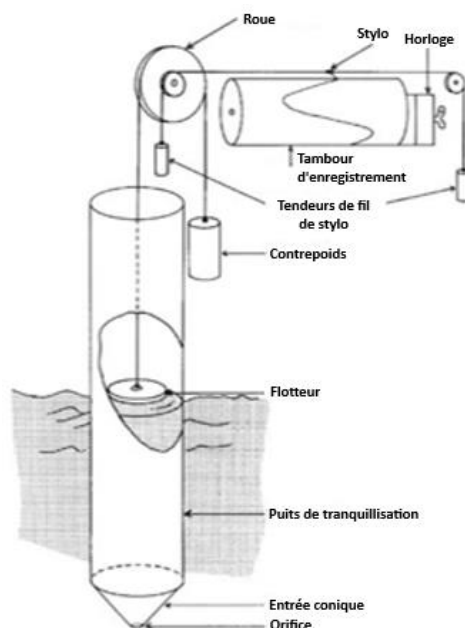


Figure 2 – Présentation du fonctionnement d'un marégraphe où le stylo entre en contact avec le papier qui tourne continuellement pour représenter la hauteur d'eau en fonction du niveau d'eau détecté

Actuellement, au LIENSs, les spécialistes souhaitant numériser un marégramme le font via un logiciel nommé NUNIEAU (NUMérisation des Niveaux d'EAU). Il s'agit d'un logiciel développé sous Matlab qui est toujours utilisé aujourd'hui et qui va transformer notre marégramme scanné en données dans un fichier texte qui va contenir la date associée au marégramme, l'heure et le niveau d'hauteur d'eau à tel moment de la journée. L'utilisateur peut choisir d'avoir une mesure toutes les 5 minutes, 15 minutes, 1 heure... Chacun fixe le seuil souhaité.

Mais ce logiciel fait face à 2 soucis majeurs : le premier étant qu'il nécessite beaucoup d'interventions humaines pour mener à bien la numérisation. C'est-à-dire que le logiciel est semi-automatique, il effectue seul la séparation du papier et de la courbe via la couleur mais c'est l'utilisateur qui doit fixer le seuil de séparation. Il effectue seul l'extraction de la courbe mais c'est l'utilisateur qui doit fixer lui-même la zone de recherche.

Le deuxième souci auquel fait face le logiciel est que lorsqu'il doit numériser un marégramme qui possède beaucoup de courbes, ça devient très vite compliqué pour l'humain délimiter la zone de recherche de la courbe souhaitée car il en est parfois trop dur de différencier qui est qui parmi les plusieurs dizaines de courbes.

1.3.Objectif du stage

L'objectif de ce stage sera donc de créer un outil/algorithme qui sera capable d'effectuer cette numérisation de marégramme. Dans un premier temps, l'objectif principal serait que cet outil puisse être capable d'automatiser tous les processus de numérisation d'un marégramme "simple" pour ensuite être capable de faire la même chose sur des marégrammes plus "complexes". À terme, cela pourra non seulement servir à gagner du temps lorsque quelqu'un souhaitera numériser un marégramme mais également pour numériser certains marégrammes qui sont trop compliqués à numériser avec le logiciel NUNIEAU et ainsi avoir une trace numérique de ces marégrammes.

2. État et analyse du problème

2.1.Présentation des données

Les marégrammes que l'on a disposition sont des marégrammes papier qui ont été scannés et sont devenus des fichiers .tif (format de fichier qui permet de stocker des images sans perte) dont certains font plus de 100 Mo.

Les marégrammes sur lesquels on va travailler possèdent à peu près tous la même structure : date de pose et de retrait du marégramme écrit en haut à la main, axe des abscisses qui se

divise en 24 heures, courbe noire ou bleu foncée sur fond rouge et blanc. Les marégrammes sur lesquels nous allons travailler capturent le niveau de marée de plusieurs jours à la suite sur la même feuille en revenant au début de la feuille pour commencer le tracé du jour suivant.

Voici un marégramme jugé “simple” (peu de courbes, pas de tâche d’encre, bien cadré...) :

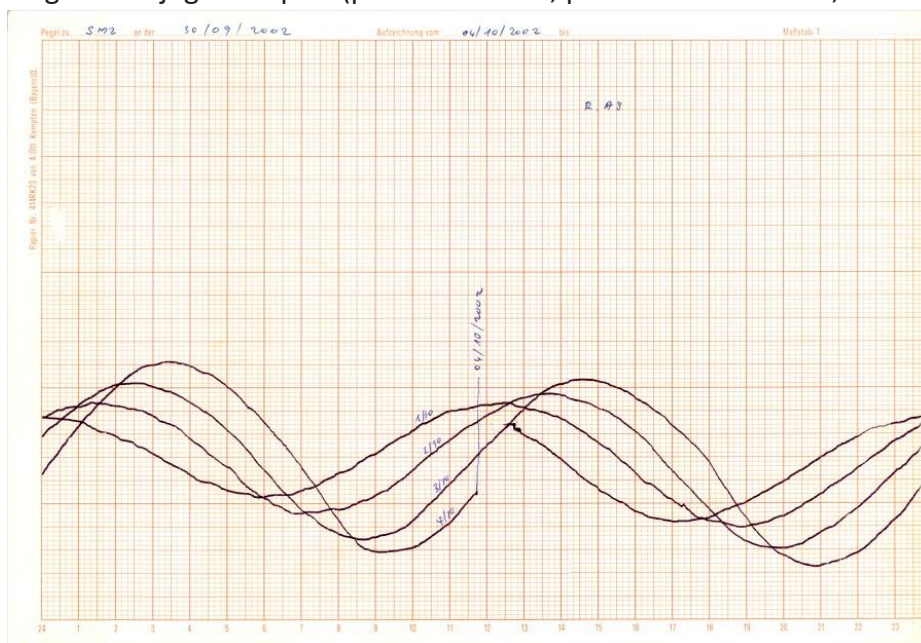


Figure 3 - Capture d’écran d’un marégramme jugé “simple” (peu de courbes, pas de tâche d’encre, bien cadré...).

Et d’autres jugés “complexes” (beaucoup de courbes, tâches d’encre, mal cadré etc)

:

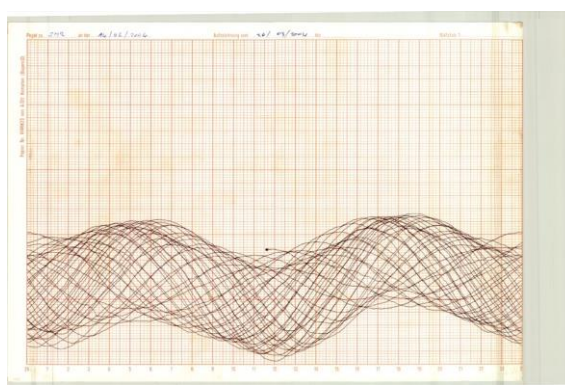
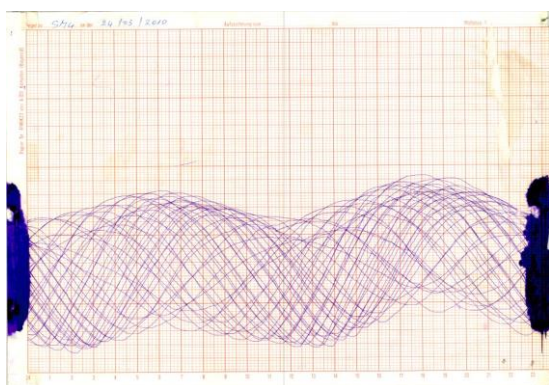


Figure 4 et 5 - Captures d’écran de marégrammes jugés “complexes” (beaucoup de courbes, tâche d’encre sur la courbe, mal cadré etc).

Certains marégrammes ont déjà été numérisées par le logiciel Nunieu dans un fichier texte à disposition. Les données qu’on y retrouve sont un relevé de données toutes les 5 minutes qui contient l’heure, la minutes et la seconde du relevé et le minimum, le maximum, le barycentre et la moyenne d’hauteur d’eau du relevé.

Année	Mois	Jour	Heure	Minute	Seconde	Minimum	Barycentre	Maximum	Moyenne	Pixels	Qualite
2000	3	24	10	35	0	119.386	123.131	126.071	122.728	940	5
2000	3	24	10	40	0	116.334	120.517	125.730	121.032	826	5

Figure 6 - Échantillon d'un fichier texte d'un marégramme numérisé manuellement.

Voici typiquement le genre de données avec lesquelles on peut travailler. Parmi ces marégrammes, il en existe certains qui sont dans des états trop spéciaux pour pouvoir être numérisés avec des outils de traitement du signal mais nous en reparlerons en temps voulu (cf 3.2 Cas limites).

2.2. Défis techniques

Le développement de notre outil va être confronté à plusieurs défis techniques.

Tout d'abord, le premier obstacle (et peut-être le plus gros) réside dans toutes les spécificités propres aux marégrammes eux-mêmes. Ce qu'on entend par là, c'est, par exemple, que faire si un marégramme est mal cadré ? Que faire s'il y a une tâche d'encre sur une courbe ? Que faire si l'encre devient moins visible qu'à l'accoutumé ?

Un deuxième point à souligner est, avant même de s'intéresser au traitement des courbes, comment isoler ces courbes du bruit ? Comment faire comprendre que je ne veux que les courbes et non pas la grille rouge ou le papier blanc ?

L'obstacle suivant est ce qui concerne le point de départ et le point d'arrivée de notre marégramme. Pour le point d'arrivée, c'est plutôt facile à gérer (vu qu'on sait qu'on est à l'arrivée lorsqu'on a plus aucun point sur lequel continuer), mais c'est une tout autre histoire pour le point de départ. Comment faire comprendre à une machine que le point de départ d'un marégramme se situe à tel endroit ?

Un autre point important à souligner est la gestion des croisements des courbes. Lorsque l'on se retrouve avec un marégramme à 40 courbes, comment faire pour que notre algorithme reste sur le bon chemin malgré toutes les sollicitations qu'il peut recevoir des autres courbes lors des multiples croisements.

Il est aussi essentiel de citer le problème qui est comment allons-nous identifier que telle courbe est associée à tel jour ?

La dernière étape technique ici est d'associer une valeur en X et Y à une valeur temporelle et une hauteur d'eau.

Un ultime défi que l'on peut citer est de faire un algorithme qui est capable de s'adapter au marégramme en face de lui, peu importe sa résolution, son nombre de courbes ou encore la

variation de teinte de l'encre. L'outil a beau fonctionner sur ce marégramme, qu'en est-il du suivant ?

2.3.Énumération des étapes pour mener à bien ce projet

Voici une liste de toutes les étapes que l'on a jugé obligatoire pour mener à bien la numérisation d'un marégramme (chaque étape sera détaillée dans la partie suivante) :

1. Identifier la date
2. Extraire la grille
3. Extraire la courbe
4. Identifier le point de départ
5. Gérer les croisements
6. Gérer le suivi de la courbe du jour suivant
7. Associer chaque valeur à un temps et une hauteur d'eau en fonction du x et du y

Pour chaque étape, on va s'assurer de tester nos méthodes sur un grand nombre de marégrammes pour s'assurer que ça marche bien pour tous et non pas pour un seul élément isolé.

3. Mise en oeuvre

3.1.Détails de la mise en oeuvre

Ces travaux ont été réalisés en langage Python et majoritairement avec la bibliothèque OpenCV, bibliothèque très utile pour faire du traitement d'image. Nous utilisons également Numpy et Scipy pour leurs nombreuses fonctions mathématiques et numériques.

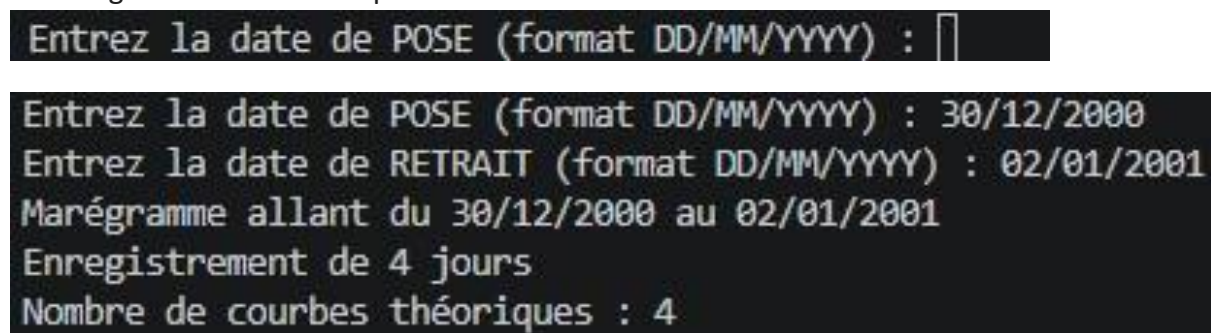
3.1.1. Identifier la date

Pour cette première étape, on se contentera d'une simple interaction avec l'utilisateur pour demander la date de pose et de retrait du marégramme. Ça a l'air quelque peu paradoxale de demander une interaction avec l'utilisateur dès le début du programme alors que l'on souhaite faire un outil qui automatise la chose, voici pourquoi. Tout d'abord, cela facilitera le reclassement temporel du marégramme par rapport aux autres. Ce qu'on entend par là, c'est que si une personne souhaite reconstruire le niveau des marées sur plusieurs mois/années, il aura juste à reprendre les données numérisées des dates qu'il aura lui-même rentré et ça

permettra de combler le vide de numérisation qu'il peut y avoir pour une région et qui dure parfois plusieurs mois voire années.

Le deuxième point à aborder est que vu la durée courte du stage, on préfère se concentrer sur des points plus importants. Il existe un moyen d'automatiser la lecture de la date avec des outils de traitement d'image type OCR (Optical character recognition) qui peuvent lire les caractères écrits à la main et donc lire la date mais comme expliqué, on préfère mettre notre temps dans des étapes plus spécifiques au problème. D'un point de vue plus technique, cette étape sert surtout pour savoir combien de courbe on est censé retrouver sur le marégramme. Exemple : si l'utilisateur indique que le marégramme va du 01/01/2001 au 07/01/2001, nous savons qu'il y a 8 jours représentés sur le marégramme et donc 8 courbes. Cela indique qu'il est impossible d'aller au-delà du suivi de 8 courbes et nous allons bloquer l'algorithme pour qu'il n'aille pas au-delà.

Voici le genre d'interaction que cela donne :



```
Entrez la date de POSE (format DD/MM/YYYY) :   
Entrez la date de RETRAIT (format DD/MM/YYYY) : 30/12/2000  
Entrez la date de RETRAIT (format DD/MM/YYYY) : 02/01/2001  
Marégramme allant du 30/12/2000 au 02/01/2001  
Enregistrement de 4 jours  
Nombre de courbes théoriques : 4
```

Figure 7 - Extrait de la méthode d'identification de la date.

3.1.2. Extraire la grille

Ce que l'on entend par extraction de la grille, c'est comme si l'on rognait le marégramme pour ne garder que ce qu'il y a à partir du bord de la grille et recadrer le tout pour que ce soit bien droit. La subtilité réside dans le fait de conserver la résolution du marégramme de base (en ayant bien sûr enlevé les pixels des bords supprimés) pour que l'échelle reste la bonne. Le fait d'avoir gardé une résolution similaire va être utile pour une étape ultérieure qui sera évoquée en temps voulu.

Pour mener à bien l'extraction de la grille, plusieurs méthodes ont été envisagées. La première étant d'extraire la grille par rapport à la forme de cette dernière. En effet, ça paraît très simple d'extraire cette grille via sa forme car nous avons juste à trouver toutes les formes présentes sur le marégramme et ensuite demander à extraire la plus grosse forme qui sera, en théorie, la grille. Mais cette méthode a montré ses limites sur les marégrammes avec tâches d'encre comme celui montré plus tôt dans le rapport (cf Figure 3). En effet, la présence de tâche au niveau du bord de la grille va venir fausser la forme de cette dernière. Ce qui fait que lorsqu'on va demander à rogner, vu que la forme détectée a été faussée, le résultat sorti n'était pas le bon.

La solution qui en a découlé reste dans le domaine géométrique mais est différente. Au lieu de venir capter la forme de la grille, on va venir capter toutes les lignes horizontales et verticales de la grille. Un premier masque contenant toutes les lignes horizontales va être créé puis un second contenant toutes les lignes verticales. Nous appliquons le premier masque sur le second et ne gardons que là où les lignes se croisent (ce sont les croisements des lignes de nos grilles). Une fois tous nos points de croisement acquis, on les plaque tous sur le même axe X et on regarde là où se trouve notre nuage de point le plus haut et le plus bas. Ce sont ces 2 nuages que l'on va considérer comme le haut et le bas de notre grille. On fait la même en plaquant tous nos points sur le même axe Y pour regarder où se trouve le nuage le plus à gauche et le plus à droite pour en déduire que ces 2 nuages sont les limites droite et gauche de notre grille. Après avoir obtenu les 4 limites de notre grille, on rogne à partir de ça.

Une rotation de la grille est aussi appliquée dans le cas où le marégramme étudié n'est pas droit. Pour se faire, nous allons tirer un trait vertical entre le bord supérieur gauche et le bord inférieur gauche de la grille détectée. À partir de là, nous pouvons voir si le trait est droit ou incliné. S'il est droit, on laisse tel quel sinon on crée une matrice de rotation d'angle inverse (opérateur mathématique qui applique l'opposé de l'angle mesuré) à l'angle détecté puis on applique une transformation affine sur cette image à partir de la matrice de rotation pour que la grille soit droite.

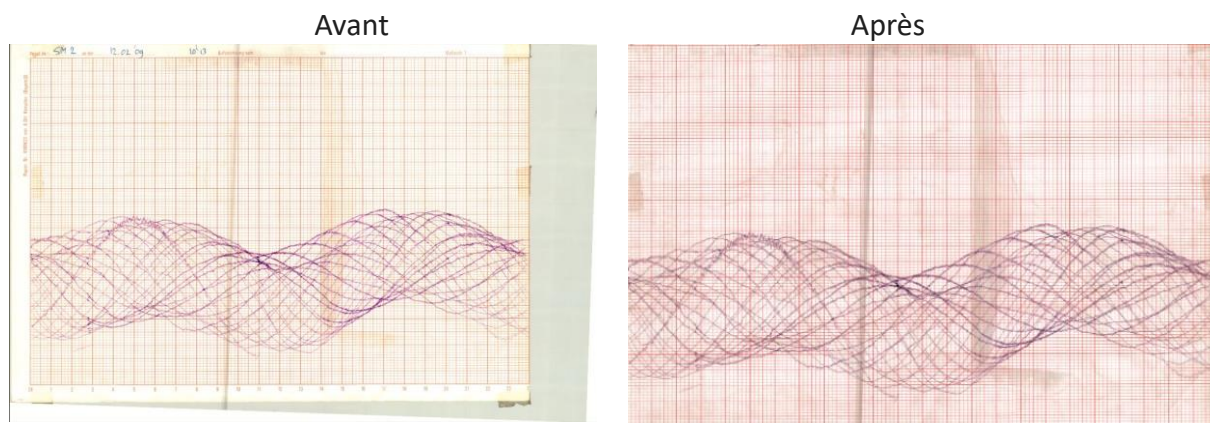


Figure 8 et 9 - Capture d'écran d'un marégramme et de sa version rognée et recadrée.

Une fois notre grille extraite, c'est sur cette image que nous allons travailler pour la suite.

3.1.3. Extraire les courbes

Pour l'extraction des courbes, c'est là que les cours de traitement d'images ont été le plus utiles. Étant donné que la majorité des courbes sont bleu foncé ou noires, la première idée qui passe par la tête est de faire une simple segmentation par couleur. C'est-à-dire que l'on sépare simplement les éléments de l'image en fonction de leur couleur. On a commencé par créer un seuil qui capture tout ce qui va du bleu foncé au noir (via la norme RGB) et on jette

le reste. Et ça marche très bien mais le problème réside dans l'existence de ce seuil. En effet, il suffit qu'une courbe soit un peu trop claire et elle ne sera pas capturée. Bien évidemment, on peut abaisser le seuil mais plus le seuil est bas et plus il y a de bruit qui est aussi capturé. Et l'autre soucis est que si l'on doit changer le seuil manuellement pour tous les marégrammes que l'on traite, on n'est pas sorti de l'auberge.

La solution pour cette partie réside également dans l'utilisation d'une méthode qui implique la couleur : l'histogramme. Un histogramme est un outil statistique qui représente la distribution des pixels de l'image en comptabilisant leur fréquence d'apparition pour chaque niveau d'intensité ou de couleur. Au lieu de fixer un seuil manuel comme avant, on laisse le programme fixer ce seuil lui-même. Pour ce faire, nous allons nous servir de la norme HSV (Teinte Saturation Luminosité) pour que le programme trouve lui-même la teinte de la courbe et ainsi capturer tout ce qui avoisine cette teinte et jeter le reste.

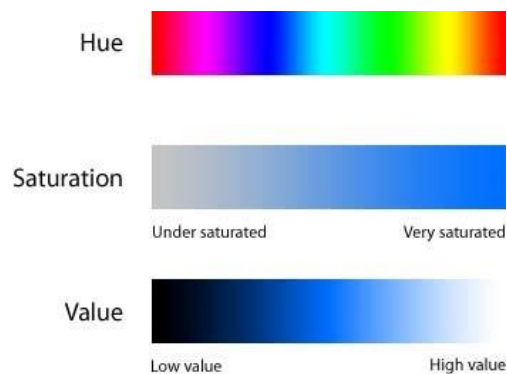


Figure 10 – Graphique montrant le raisonnement derrière la norme HSV utilisée (Source : <https://jaalsblog.wordpress.com>).

Plus précisément, nous effectuons un premier seuillage sur la luminosité (Value). Cela permet d'exclure le papier blanc en fond. Ensuite, comme on peut le voir sur le graphique de la figure 9, la teinte (Hue) du rouge (la grille) se trouve aux extrémités des degrés de teinte. On peut donc facilement dire que l'on ne veut pas de ces teintes extrêmes. La dernière étape réside dans le fait de demander à notre programme quelle teinte est la plus présente dans ce qu'il reste. Vu qu'il ne reste plus que la teinte de la courbe (bleue/noire), ce sera forcément celle-là qui sera nommée comme teinte dominante. Il ne reste plus qu'à demander de capturer tout ce qui avoisine cette teinte pour reconstruire les courbes dans l'espace binaire (espace où un pixel vaut 0/noir ou 1/blanc) et voici ce que l'on obtient :

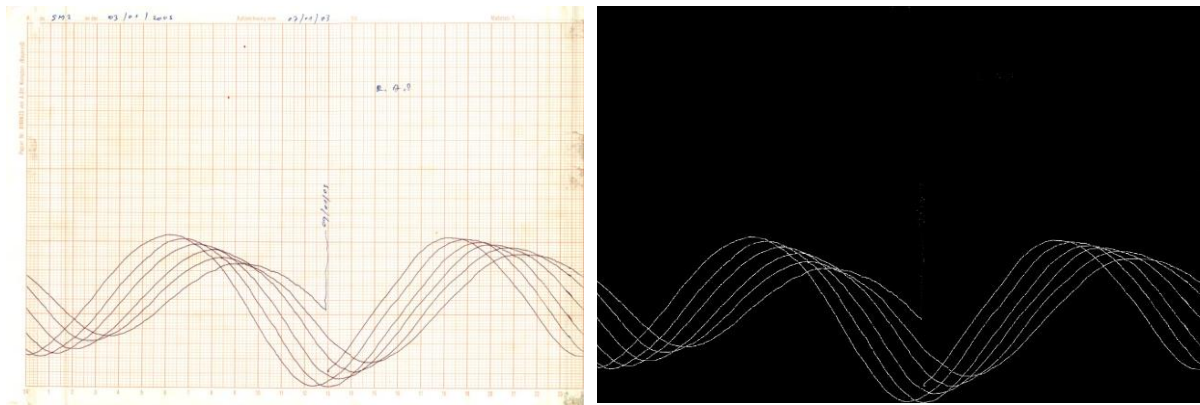


Figure 11 et 12 – Représentation d'un marégramme avant et après extraction de la courbe via la norme HSV (Hue Saturation Value) dans l'espace binaire.

Une fois la courbe extraite, on applique une dilatation pour faire grossir la courbe et boucher quelques trous dans les courbes puis on termine par une squelettisation. C'est une technique de morphologie mathématique qui va tailler, petit à petit, les bords des courbes pour qu'elles ne fassent plus qu'un pixel d'épaisseur. Cette squelettisation va être utile pour qu'un seul pixel ne soit associé qu'à une valeur temporelle et une hauteur d'eau et évite de faire un choix lorsqu'on se trouve à un moment de la courbe qui aurait plusieurs pixels pour la même valeur temporelle sans la squelettisation. Ce marégramme reconstruit dans l'espace binaire (cf Figure 11) est la version du marégramme sur laquelle les étapes restantes vont travailler.

D'un point de vue validation de l'étape, il n'existe pas vraiment de métrique toute prête pour valider que l'extraction s'est bien effectuée à 100%. Seule une vérification visuelle entre le marégramme de base et sa version binaire peut servir à voir si c'est bien reconstruit. Concevoir une métrique est possible mais la durée du stage impose de ne pas trop s'attarder. Une vérification a quand même été effectuée sur une vingtaine de marégrammes différents pour s'assurer du bon déroulement du procédé.

3.1.4. Trouver le point de départ

Quel meilleur endroit pour le suivi des courbes d'un marégramme qu'en commençant par le début. Cette recherche du point de départ est l'un des 2 défis techniques les plus compliqués à résoudre car lorsqu'on se retrouve avec un point de départ en plein milieu d'une dizaine de courbe, c'est parfois compliqué pour un humain de le trouver alors imaginez faire comprendre cette logique à une machine.

La première idée envisagée est simplement de faire comprendre à notre algorithme qu'un point de départ d'une courbe est un point qui n'a forcément pas de voisins à sa gauche. L'une des choses qui nous permet facilement de mettre en œuvre cette logique est que les marégrammes ont tous été posés à la main par un humain et donc posés à des horaires normaux. On n'aura jamais de point de départ en plein milieu de la nuit vers minuit (voir Figure 10). Ce qui veut dire que l'on peut d'ores et déjà éliminer tous les débuts de courbe à

gauche dans le marégramme qui peuvent être interprétés par le programme comme des points de départ. Ça nous permet d'éliminer beaucoup de candidats et dans pas mal de cas, le dernier candidat restant sera le vrai point de départ.

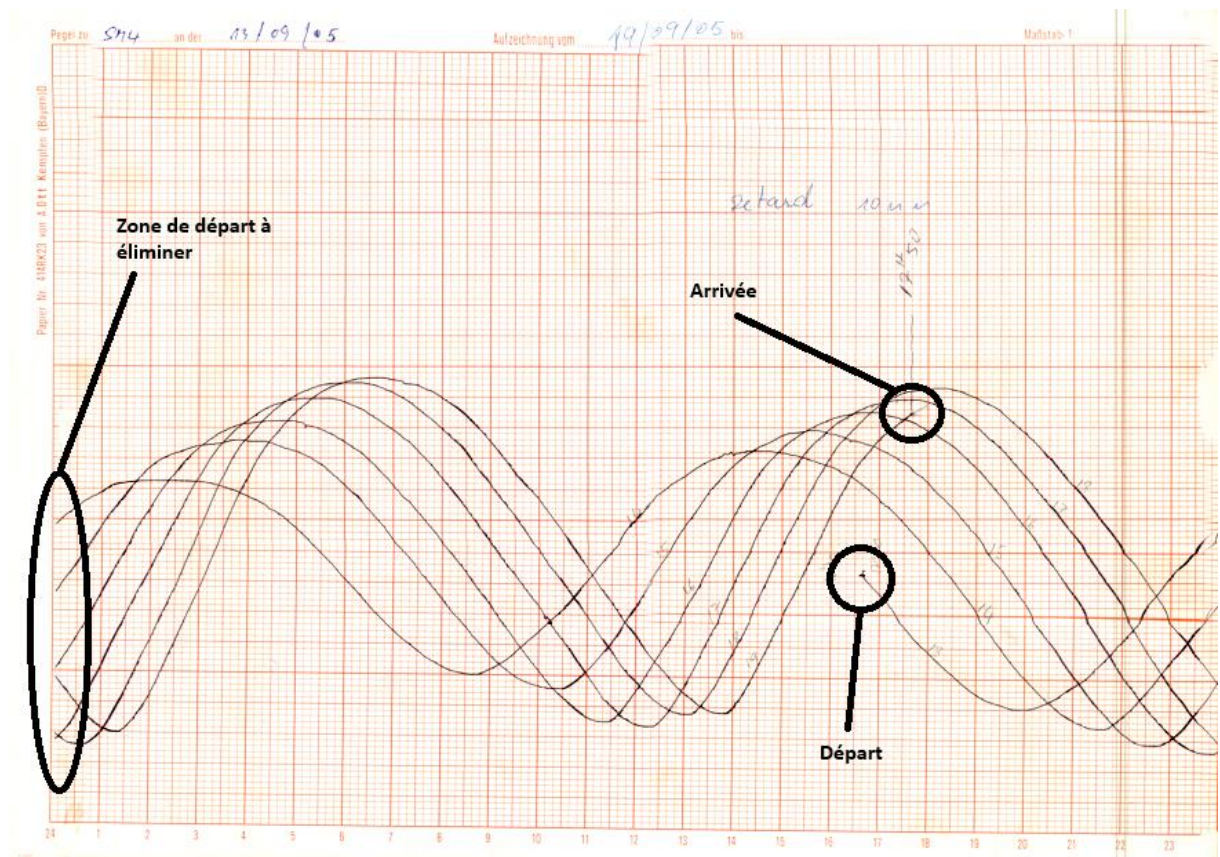


Figure 13 – Décryptage des zones de départ impossible d'un marégramme dû à leur dépendance de la pose humaine (impossible d'être posé à minuit).

Pour montrer que cette logique marche dans certains cas, voici un échantillon d'un test réalisé au début du stage.

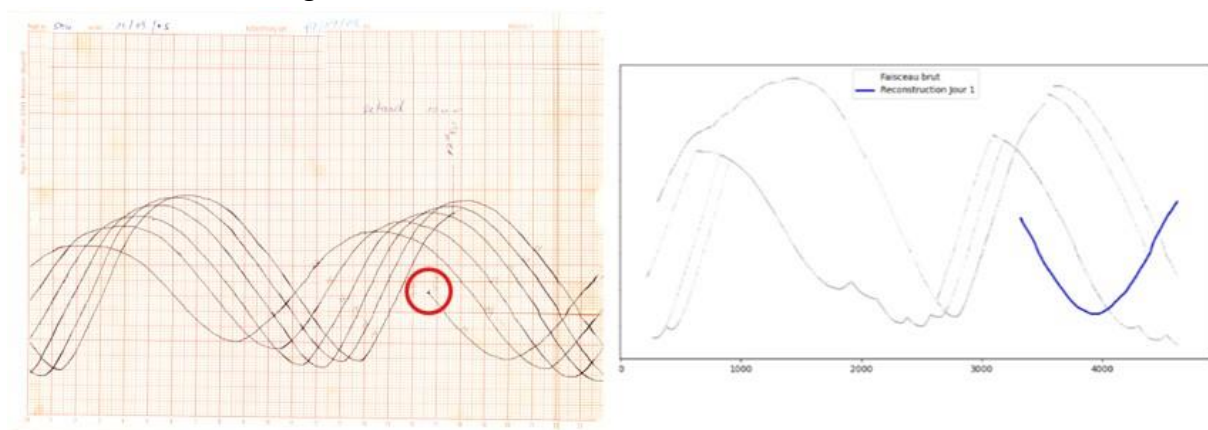


Figure 14 et 15 – Capture d'écran d'un marégramme avec point de départ entouré en rouge et reconstruction de la courbe du premier jour (courbe bleue) pour montrer que le point de départ est le bon.

Le point de départ (vers 16h30) est facilement lisible car il est complètement seul et n'est dérangé par personne autour. La mise à l'échelle en Y est différente mais l'important ici est de se focaliser sur ce point de départ trouvé.

On peut voir que notre algorithme à bel et bien détecté le point de départ de notre marégramme. Là où cette logique pêche, c'est lorsque l'on se retrouve avec un point de départ qui chevauche le tracé d'une autre courbe. En effet, si un point de départ est chevauché ou même ne serait-ce qu'effleuré par une autre courbe, ça veut dire qu'il possède un voisin dans sa zone gauche et donc l'algorithme ne le reconnaîtra plus et tout tombe à l'eau.

D'un point de vue validation, au-delà de la simple validation visuelle, comment s'assurer que l'on a bien le bon résultat ? En utilisant le fichier texte d'un marégramme déjà numérisé manuellement, nous pouvons savoir à quel heure/minute se situe le point de départ, nous faisons ensuite un simple produit en croix pour voir si la coordonnées X du point de départ trouvé correspond à l'heure réelle. En d'autres termes, imaginons un marégramme dont on sait grâce à la numérisation manuelle que le point de départ est à 15h (équivalent à 900 minutes). Le nombre total de minutes dans une journée de 24h est de 1440 minutes. La résolution totale du marégramme en X est de 8500 pixels. Mon programme me renvoie qu'il a trouvé un point de départ en $X = 5000$. On effectue un produit en croix pour voir si l'on obtient bien les 900 minutes à la fin. Si le résultat est bon sur plusieurs marégrammes, on estime que la logique est bonne sinon, ça veut dire qu'il y a des choses à corriger.

1440	8500
?	3500

3.1.5. Gérer les croisements

Voici le deuxième plus gros défi technique de ce sujet. Lorsque 2 courbes se croisent, comment peut-on faire comprendre que la suite du tracé à suivre est cette courbe et non pas l'autre.

Avant même de s'intéresser à comment nous allons traverser ces croisements, il faut d'abord savoir comment nous détectons un croisement. Pour ce faire, à chaque fois que nous avançons d'un pixel, une zone va se déployer autour de ce pixel qui va venir compter, dans l'espace binaire, tous les pixels blancs dans la zone. Si le nombre de pixels blancs excède le seuil, alors on en déduit que l'on fait face à un croisement. Lorsqu'on a affaire à un croisement, la logique s'enclenche automatiquement.

L'idée première a été d'utiliser la régression linéaire. Cette méthode étudie les derniers points de la courbe avant un croisement et va en déduire une tendance. Cette tendance va nous servir à trouver quelle courbe on va ensuite emprunter lors du croisement en fonction du score des candidats. Il faut imaginer que c'est comme si on trace une ligne au niveau du croisement, cette ligne représente le chemin géométrique le plus probable que la courbe

doit suivre si elle conserve son inertie physique naturelle (cf Figure 15) et que l'on calcule ensuite le score par rapport à cette ligne. Pour calculer le score, on calcule 2 sous-scores : la distance euclidienne entre le dernier pixel du croisement et tous les pixels candidats que l'on additionne ainsi que l'écart de la hauteur entre le pixel candidat et là où il devrait se trouver selon la ligne tracé.

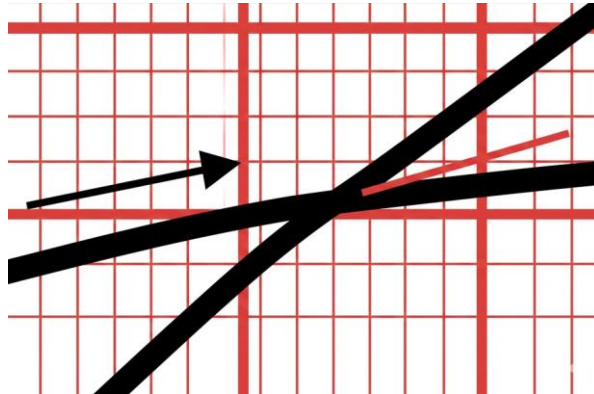


Figure 16 : *Reproduction d'un croisement des courbes avec flèche noire pour indiquer la courbe sur laquelle on est et ligne rouge pour indiquer la pente prédite grâce à la régression linéaire.*

Si on réutilise l'exemple des figures 13 et 14 en zoomant sur les parties qui nous intéressent, voici ce que l'on obtient pour la reconstruction de la courbe du jour 1 de ce marégramme en intégrant la logique de gestion des croisements :

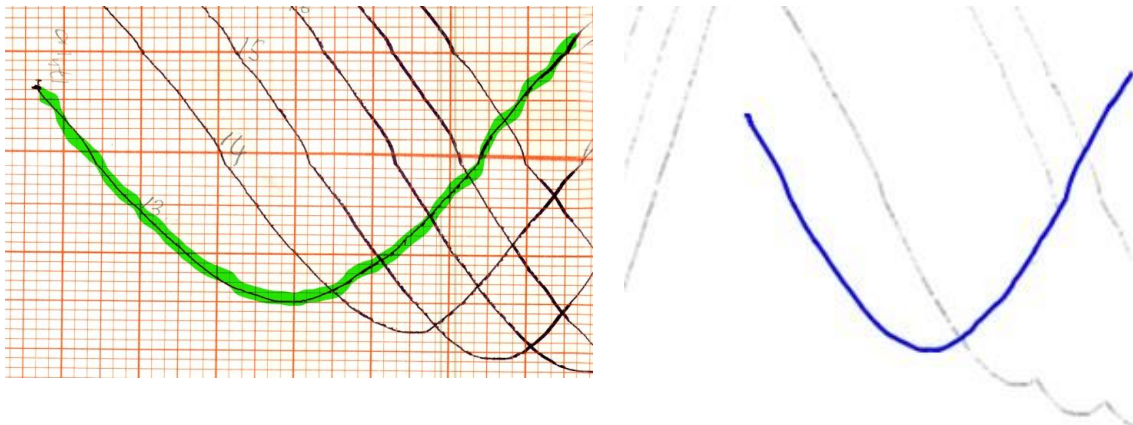


Figure 17 et 18 - *Représentation du suivi de la courbe incluant la gestion des croisements.*

Autre exemple sur le marégramme utilisés sur les figures 10 et 11 :

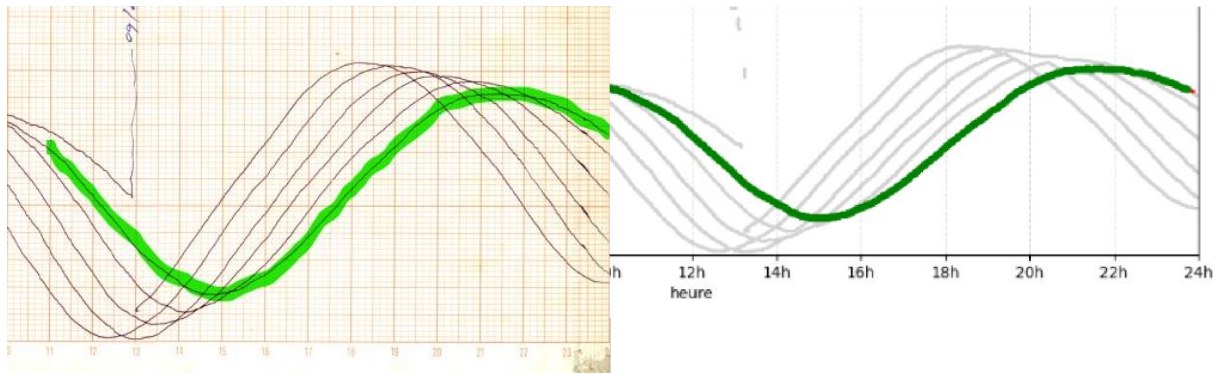


Figure 19 et 20 - Autre représentation du suivi de la courbe incluant la gestion des croisements

Dans certains cas, cette gestion des croisements va passer mais il y a une chose qui vient corrompre cette gestion des croisements, c'est la squelettisation appliquée lors de l'extraction des courbes. Cette dernière vient fusionner les courbes lors des croisements ce qui vient fausser le croisement (cf Figure 20).



Figure 21 - Focus sur le croisement des courbes dans l'espace binaire faussé par la squelettisation

Donc dans certains cas, le calcul de la pente sera faussé et l'algorithme choisira la mauvaise courbe.

3.1.6. Gérer le suivi de la courbe du jour suivant

Pour cette logique, c'est l'une des plus intuitives à implémenter. Lorsque l'on arrive au bout du suivi de la courbe du jour en cours (c'est-à-dire lorsqu'on atteint le dernier pixel disponible par rapport à la largeur de la grille), l'algorithme procède ainsi :

1. Stocker dans une variable la coordonnée Y du dernier point valide de la journée passée.
2. Repositionner virtuellement son curseur horizontal au départ de l'image ($X = 0$).
3. En partant de cette colonne $X = 0$, il cherche le pixel de courbe disponible dont la hauteur est la plus proche du Y précédemment mémorisé. En raison de la continuité physique de la marée, le niveau de la mer ne peut pas varier de manière instantanée, le début de la courbe du jour suivant est donc factuellement le tracé le plus proche de cette hauteur.

Bien évidemment, un seuil d'erreur existe. On ne demande pas à trouver un début de courbe qui correspond parfaitement à la coordonnée captée à la fin de la courbe précédente. On

effectue une sorte de scan de la zone qui est censé accueillir la suite du suivi pour voir si la suite se trouve bien là.

À chaque fois qu'un passage au jour suivant est effectué, une variable est incrémentée pour s'assurer que notre nombre de jours est raccord avec le nombre de jours indiqué par l'utilisateur lors de la déclaration des dates. Si on calcule qu'il y a plus de jours calculés que ce qui est indiqué par l'utilisateur, alors on peut en conclure qu'il y a une erreur quelque part.

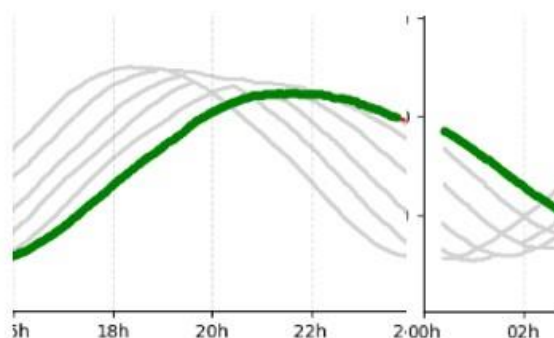


Figure 22 - Montage présentant comment se déroule le suivi du jour suivant à partir de la coordonnée Y du dernier point de la courbe précédente.

3.1.7. Convertir chaque valeur en un temps et une hauteur d'eau

Voici l'étape finale de la numérisation : transformer ces courbes en chiffres exploitables. Passer d'une valeur en X et en Y à une valeur en temps et en hauteur d'eau.

En ce qui concerne la transformation de valeur en X en données temporelles, l'idéal est de reprendre la même idée que NUNIEAU, c'est-à-dire de prendre une mesure toutes les 5 minutes. Cela permettra non seulement de reconstruire plus fidèlement le marégramme mais surtout ça nous servira à comparer directement nos résultats avec les résultats déjà numérisés manuellement dont les mesures sont aussi espacées de 5 minutes.

Du côté de la transformation de valeur en Y en hauteur d'eau, c'est un peu particulier. En effet, si l'on regarde bien l'axe des ordonnées, on voit qu'il n'y a aucune mesure qui indique explicitement où se situe 1 mètre de hauteur d'eau, 2 mètres etc. Le secret réside dans le comptage des cases. Une case représente 4 centimètres de hauteur d'eau réelle.

Pour s'en assurer, faisons une vérification rapide avec un marégramme dont les données ont déjà été numérisées.



Figure 22 - Zoom sur une mesure du jour 1 d'un marégramme à 12h10 (entouré en bleu).

2000	3	24	12	10	0	57.770	59.644	61.414	59.592	406	5
------	---	----	----	----	---	--------	--------	--------	--------	-----	---

Figure 23 - Equivalent numérique de la mesure évoquée précédemment avec valeur finale d'hauteur d'eau (moyenne) surligné.

Une case équivaut à 4 cm d'hauteur d'eau, il y a 15 cases. $15 * 4 = 60$ centimètres d'hauteur d'eau. Le calcul est bon.

Dû au manque de temps, cette partie n'a pas pu être implémenté. Voir 4.2 Évolution du projet.

3.2. Cas limites

Il existe plusieurs types de cas où les marégrammes sont tellement spéciaux qu'il en devient impossible de travailler avec, voici quelques exemples.

Les marégrammes dont la structure est faussée par un élément extérieur. C'est exactement le cas du marégramme ci-dessous (cf Figure 23) où un deuxième stylo a dû complètement déborder sur le marégramme ce qui va créer cet autre tracé bizarre. Ici, l'extraction de la grille est impossible avec notre méthode car la forme et les croisements de la grille sont faussés par l'autre tracé, l'extraction de la courbe est aussi compliquée car la teinte et la couleur est similaire entre le vrai et le faux tracé.

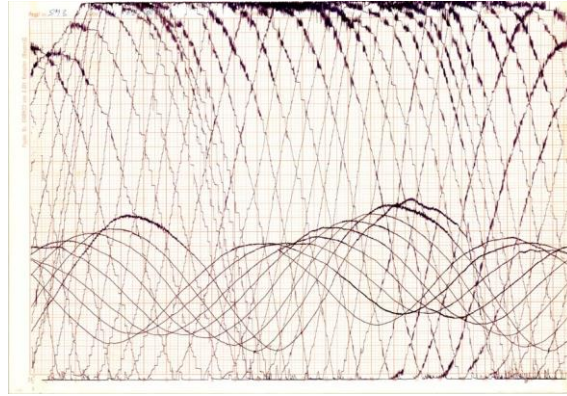


Figure 24 - Affichage d'un marégramme sur lequel il est pratiquement impossible de travailler dû aux nombreuses courbes parasites.

Les marégrammes dont l'encre est trop claire pour être captée. Ici, l'encre est tellement claire que l'on dirait qu'il n'y a même pas de courbe et pour l'algorithme, c'est comme si la courbe était confondue avec le papier.

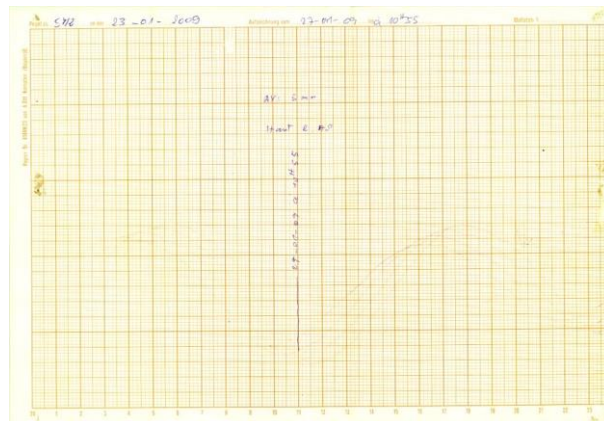


Figure 25 - Affichage d'un marégramme sur lequel il est pratiquement impossible de travailler dû à l'encre beaucoup trop claire qui rend l'extraction des courbes plus laborieuse.

L'alternative envisagée est d'utiliser un seuillage adaptatif. Cela consiste à tracer une zone de X pixels autour de tous les pixels de l'image. Si le pixel a une teinte plus sombre que la moyenne de sa zone, alors il devient blanc dans l'espace binaire, sinon noir. Cela permet de capter toutes les spécificités de l'image mais le problème étant que ça capte tellement bien les spécificités qu'il y avait trop de bruit sur le rendu binaire. On demande donc à notre algorithme de supprimer toutes les lignes droites de plus de 10 pixels, ce qui va supprimer les lignes de la grille et les différentes lignes droites non souhaitées pour ne laisser que les courbes.

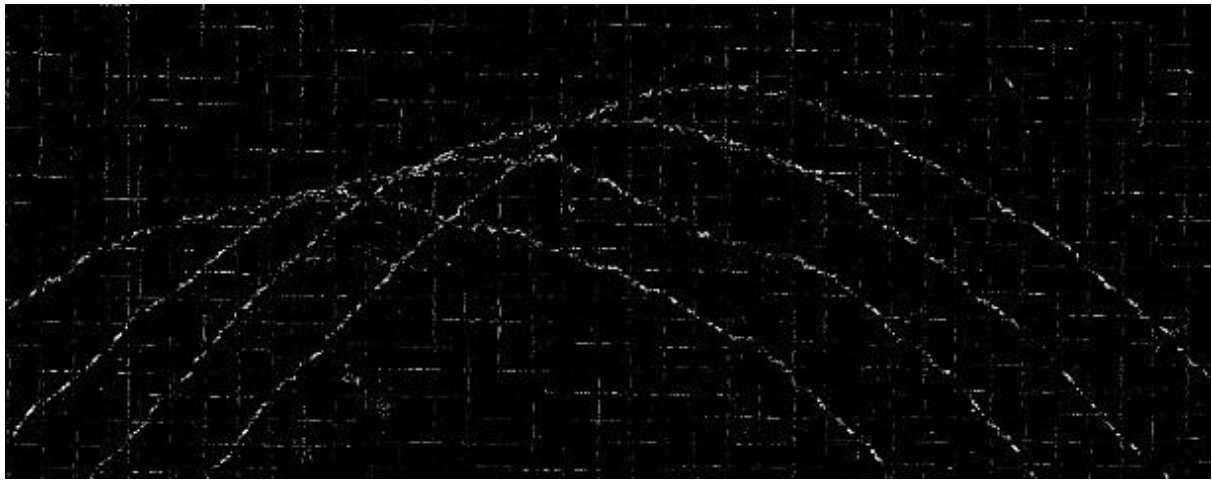


Figure 26 - *Vue rapprochée de la reconstruction des courbes avec seuillage adaptatif et beaucoup de bruit.*

Or, quand bien même on essaye de supprimer un maximum de bruit, on voit bien qu'il en reste beaucoup trop pour pouvoir vraiment travailler sur nos courbes. On voit également que les courbes sont très saccadées. Bref, nos outils ne permettent pas de satisfaire l'entièreté des problèmes.

4. Conclusion

4.1. Bilan des apports du projet

Au-delà des résultats purement techniques, la première réussite réside dans la démarche méthodologique adoptée. Nous avons réussi à analyser le problème en décortiquant chacune des étapes pour à la fois établir un plan d'action qui va mener à bien cette numérisation de marégramme mais on vient également apporter une solution pour chacune des étapes. Quand bien même ce n'est pas encore parfait, le gain de temps est réel et ça pose une bonne base pour la suite.

D'un point de vue technique, notre outil a permis plusieurs choses. Dans un premier temps, nous avons réussi à créer un algorithme capable d'extraire la grille d'un marégramme sans pour autant en avoir altéré la résolution grâce aux croisements de cette grille qui permettent d'en conclure les bords de cette dernière. Nous avons également su créer un outil capable d'extraire de lui-même les courbes grâce à un histogramme HSV qui établit automatiquement la teinte des courbes recherchées.

Les bases ont été posées concernant les logiques de recherche du point de départ et de gestion des croisements. Cette première fonctionnant grâce à une recherche des voisins de gauche des potentiels points de départ et cette dernière fonctionnant avec la régression

linéaire pour étudier les derniers points avant un croisement et ainsi en déduire la tendance à suivre.

Nous avons également implanté le nécessaire pour gérer le suivi des courbes des jours suivants en utilisant la logique qui permet de retrouver la courbe du jour suivant grâce à la coordonnée Y du jour précédent.

4.2. Évolution du projet

Le projet n'étant pas finalisé à 100%, il existe des points qui peuvent être améliorés voir complètement remaniés.

On peut commencer en parlant de la première étape qui consiste à rentrer manuellement les dates du marégrammes. On peut l'améliorer de deux façons : la première idée peut-être de garder cette logique de rentrer manuellement la date du marégramme mais en essayant de faire autre chose qu'utiliser la console. On peut par exemple créer une petite fenêtre graphique qui s'ouvre quand on lance le programme et qui permet une interaction plus conviviale qu'une console mais ce n'est qu'un changement au niveau de la forme. La deuxième façon d'améliorer ceci réside dans le changement du fond. Au lieu de laisser l'utilisateur rentrer à la main les dates, on peut utiliser des outils d'OCR qui liront automatiquement la date (lorsqu'elle est disponible bien sûr).

Parlons ensuite des parties extraction de grille et de courbes. Du point de vue extraction de grille, il n'y a pas d'outils de traitement de l'image qui fera une avancée majeure comparé à ce que nous avons déjà mis en place. Concernant les cas limites, une approche Machine Learning peut être envisagé. Il s'agit d'une approche de l'IA consistant à entraîner un algorithme pour exécuter une tâche qu'il sera capable de résoudre de manière autonome, même face à de nouvelles données inconnues. Cette approche combinée avec des données entraînées à reconnaître des grilles et des courbes (foncées et claires) pourrait venir à bout de ce problème. Cette approche permettrait à la machine d'apprendre ce qu'est une grille et une courbe et ainsi effectuer les opérations d'extraction à partir de cela.

Le Machine Learning pourrait également être utile pour les deux points qui coïncident, la recherche du point de départ et la gestion des croisements. Une intelligence artificielle pourrait apprendre à reconnaître ce qu'est un point de départ et ainsi le détecter, peu importe s'il est isolé, entouré d'autres courbes voire carrément sur le tracé d'une autre courbe. Pour la gestion des croisements, une IA peut regarder l'histoire de la courbe sur une longue distance et ainsi en prédire le tracé futur et faire les bons choix. Si l'on ne veut pas de Machine Learning pour trouver le point de départ, on peut aussi utiliser une approche plus traditionnelle qui consiste à demander à l'utilisateur de sélectionner lui-même le point de départ du marégramme en cliquant sur ce dernier dans une nouvelle fenêtre. Certes on perd le côté automatique mais disons que c'est une méthode qui peut être discuté si l'on veut rester intégralement sur des outils de traitement d'image.

Du côté de la logique pour transformer nos valeurs X et Y en données temporelles et d'hauteur d'eau, voici comment on aurait réfléchi si le temps l'avait permis. Comme indiqué précédemment (cf Mise en oeuvre), l'idéal serait d'avoir une mesure toutes les 5 minutes. Vu que l'on connaît d'avance la largeur en pixel de notre marégramme (grâce à la fonction `shape()`) et que l'on sait qu'il est sur 24 heures (1440 minutes), un simple calcul permet d'obtenir le pas d'échantillonnage :

$$\text{Pas en pixels (5 min)} = \frac{\text{Largeur_grille}_{\text{pixels}}}{1440} \times 5$$

Grâce à ça, on sait combien de pixels sépare chacun de nos intervalles de 5 minutes. C'est en réalité un simple produit en croix qui converti la largeur totale en largeur pour 5 minutes. Et ça nous permet également de savoir où se situe temporellement sur le marégramme. Si j'ai fait 12 itérations de 5 minutes, je sais que 2 heures sont passées.

Et côté hauteur d'eau, on sait qu'une case vaut 4 centimètres d'hauteur d'eau et que nos marégramme font 125 cases de haut. On en conclut qu'un marégramme peut représenter jusqu'à 5 mètres d'hauteur eau mais surtout que la hauteur en pixels d'une grille est :

$$\text{Taille d'une case}_{\text{pixels}} = \frac{H_{\text{grille}}}{125}$$

Et on peut ainsi en déduire la hauteur d'eau réelle grâce à cette formule :

$$\text{Hauteur d'eau réelle (en cm)} = \frac{Y_{\text{local}} \times 125}{H_{\text{grille}}} \times 4$$

Où H_{grille} représente la hauteur totale de la grille en pixels, Y_{local} représente la hauteur en pixel du point sélectionné (il faut bien faire attention car, en traitement d'image, le 0 est en haut à gauche donc on fait $H_{\text{grille}} - Y_{\text{local}}$ pour obtenir un Y_{local} dans le bon sens) et 4 est le nombre de centimètres d'hauteur d'eau d'une case.

Pour vérifier que l'on obtient bien les bons chiffres, une RMSE peut être effectuée entre les données d'un marégramme déjà numérisé et les données que nous avons obtenues grâce à notre algorithme sur ce même marégramme. Un score en sera tiré qui permettra de savoir si la numérisation a été menée à bien ou non. Effectuer ce test sur des marégrammes différents confirmera si l'algorithme est performant et prêt à être mis en oeuvre.

Une fois les mesures obtenues, on peut stocker le tout dans un fichier texte ou un CSV et laisser libre recours à chaque personne de faire ce qu'il souhaite de son marégramme numérisé.

4.3. Bilan personnel

Lors de ce stage, j'ai pu approfondir les connaissances de traitement d'images que j'ai pu apprendre lors de cette 3e année de licence, notamment dans les cours "Traitement du signal pour les objets intelligents" du semestre 5 et "Vision embarquée et intelligence artificielle" du semestre 6 ou reprendre des concepts du cours "Analyse de données " des semestres 4 et 5. J'ai pu découvrir certaines logiques qui m'étaient peu connues comme les arbres à K dimensions qui m'ont été très utiles pour effectuer la recherche de voisins d'un point. Le fait de travailler sur de vraies données historiques permet de se rendre compte du travail effectué en amont rien que pour éliminer le bruit et obtenir une base saine sur laquelle travailler. Là où, en cours, les données utilisées sont déjà quasiment vierges de bruit, cela permet d'apporter un autre regard sur les travaux effectués en traitement d'image et même dans d'autres domaines.

En ce qui concerne la partie purement méthodologique, j'ai appris à décortiquer un problème de A à Z pour en tirer le meilleur échelonnement possible des tâches à faire. Une grande part d'autonomie m'a été laissée lors de ce stage, ce qui fait que j'ai pu choisir les propres méthodes sur lesquels je voulais travailler et j'ai aussi dû apprendre à vulgariser mon travail pour le montrer à des personnes plus novices sur ce thème. Cela me force aussi à faire comprendre pourquoi je peux privilégier telle ou telle idée. Un autre point important que je souhaite mentionner, c'est d'avoir appris à accepter l'échec. On n'obtient pas des résultats satisfaisants du premier coup et la majeure partie des succès sont construits grâce aux échecs. L'important est de comprendre pourquoi ça ne marche pas, poser des hypothèses et tester des solutions.

Mon objectif est d'orienter ma carrière vers le domaine de la Data. Quand bien même ce stage s'oriente plus sur du traitement d'image, il s'inscrit complètement dans mon parcours universitaire qui est aussi orienté traitement d'image, traitement du signal et intelligence artificielle. C'est une double spécialisation que je peux mettre en avant. Non seulement je peux travailler avec la donnée mais je suis capable de la nettoyer et de la comprendre en amont.

4.4. Ouverture

Comme évoqué précédemment (cf 4.2 Évolution du projet), le futur de ce projet réside clairement dans l'utilisation du Machine Learning pour parvenir à combler les trous que les outils de traitement de l'image ne peuvent boucher (ou très laborieusement). C'est d'ailleurs ce qui prévu de faire dans les semaines restantes après le dépôt de ce rapport.

À plus long terme, la finalisation d'un tel outil dépasse le cadre d'un simple script informatique. Elle ouvre la voie à d'autres développeurs souhaitant travailler sur cet outil capable de traiter automatiquement et par lots des décennies d'archives marégraphiques à

travers le monde. En transformant ces documents papier oubliés en fichiers de données, ce projet s'inscrit directement dans une démarche d'utilité publique. Les séries temporelles ainsi extraites permettront d'alimenter les bases de données océanographiques qui sont, pour certaines, encore vides ou remplies de trous temporels, offrant un outil précieux pour affiner les modèles de prévision de l'élévation du niveau des mers et mieux comprendre l'impact du changement climatique sur nos littoraux.

Sources :

- Site du Shom : <https://www.shom.fr/>
- Page du Shom sur le REFMAR : <https://refmar.shom.fr/>
- Site du LIENSs : <https://lienss.univ-larochelle.fr/>
- Site du MIA : <https://mia.univ-larochelle.fr/>
- Site de W3schools : <https://www.w3schools.com/>
- Documentation de NumPy : <https://numpy.org/>
- Documentation de SciPy : <https://scipy.org/>
- Documentation de OpenCV : <https://docs.opencv.org/4.x/pages.html>
- Site de Expert Python : <https://www.expertpython.fr/>
- Cours Moodle d'Analyse de Données (S4 et S5), de Traitement du signal pour les objets intelligents (S5), de Programmation fonctionnelle (S6) et de Vision embarquée et Intelligence artificielle (S6)
- Présentation de DJEUMENI NOUBISSIE Lucrèce